POST MANAGEMENT SYSTEM

MERN Stack Development — Complete 4-Member Group Project Plan

Project Type	MERN Stack Web Application
Project	Post Management System
Team Size	4 Members
Primary Goal	Authenticated system for creating, managing and interacting with posts
Development Approach	Feature-based division with one integration lead

Purpose of this document

This document is a systematic implementation and demonstration plan for the assigned Post Management System. It defines the project scope, explains the group-division problem, establishes the recommended architecture and database structure, and assigns clear responsibilities to all four members so the final project can be developed and merged without unnecessary conflicts.

1. Assignment Problem

The class instruction is:

“As discussed in class and already demonstrated in lab classes, all students are instructed to check the list of projects assigned group wise for MERN Stack dev. You'll be assigned with dates after the puja vacation for this project demonstration where group wise you've to elaborate your work & respective understanding. No change of groups are to be entertained. Start working. Best wishes.”

The assigned project for this group is a **Post Management System**. The group must build a working MERN application and, during demonstration, explain both the implemented work and the underlying MERN concepts.

2. Project Objective

The system should allow registered users to authenticate, create and manage their own posts, interact with other posts through likes and comments, search and browse content, and view a personal dashboard. An administrator should be able to manage users and moderate posts.

- **Authentication:** registration, login, logout, JWT-based protected access and user profile.
- **Post management:** create, read, update and delete posts.
- **Interaction:** likes, unlikes and comments.
- **Discovery:** search, filtering and optional pagination.
- **Dashboard:** personal post and engagement information.
- **Administration:** user management and post moderation.
- **Demonstration:** every member must understand the complete request flow, not only their assigned files.

3. The Main Group-Work Problem

A natural first idea is to divide the four MERN technologies between four people: Member 1 = MongoDB, Member 2 = Express, Member 3 = React and Member 4 = Node.js. This looks balanced, but it is not a good development division.

Why technology-wise division causes conflicts

Problem	Why it happens
MongoDB is not isolated	Database schemas must match backend models, controllers and frontend data requirements.
Express and Node.js overlap	Express is a backend framework running on Node.js, so separating them creates unclear ownership of the same backer
React depends on the API	The frontend cannot be finalized until API routes, request format and response format are agreed.
One feature touches all four technologies	Creating one post can involve React, Axios, Express, Node.js, Mongoose and MongoDB.
Higher Git conflict risk	Multiple members are likely to modify the same backend configuration, models or integration files.
Weak demonstration ownership	A member may understand one technology but not how a complete project feature works from frontend to database.

Example: creating a post

Layer	Responsibility
React	Collect title/content from the user and submit the form.
Axios/API communication	Send the request to the backend.
Express	Receive and route the HTTP request.

Layer	Responsibility
Node.js backend logic	Execute validation and business logic.
Mongoose	Map post data to the MongoDB model.
MongoDB	Store the post document.
Response	Return the result to React and update the interface.

Conclusion: Divide the project by functionality/features rather than assigning one MERN technology to each person.

4. Recommended Project Architecture

The application follows the standard MERN flow:

Layer	Technology	Purpose
Frontend	React	Pages, components, forms, navigation and user interface.
HTTP/API communication	Axios	Communicates between React and backend.
Backend/API	Node.js + Express	Routes, controllers, authentication middleware and business logic.
Data modeling	Mongoose	Defines MongoDB schemas and handles database operations.
Database	MongoDB	Stores users, posts and comments.

Overall flow: User → React UI → API request → Express route → controller/business logic → Mongoose → MongoDB → response → React UI

5. Complete Project Structure

Use one GitHub repository with separate client and server areas.

Area	Main contents	Purpose
client/	components, pages, services, context, hooks, styles, routing	React frontend
server/	config, controllers, middleware, models, routes, utils	Node/Express backend
server/models/	User, Post, Comment	Database structures
server/controllers/	auth, user, post, comment, like, admin	Application logic
server/routes/	auth, user, post, comment, like, admin	API endpoints
server/middleware/	authentication, admin, errors, optional upload	Cross-cutting backend controls
client/services/	API, auth, post, comment, admin services	Frontend-backend communication
docs/	API, database and project documentation	Group documentation
Root	README, .gitignore, package configuration	Project-level configuration

6. Main Application Pages

Page	Purpose	Access
Home	Browse posts and search content	Public
Register	Create account	Public
Login	Authenticate user	Public
Profile	View/update profile	Logged-in
Dashboard	Personal statistics/activity	Logged-in
My Posts	View own posts	Logged-in
Create Post	Create post	Logged-in
Edit Post	Modify own post	Logged-in

Page	Purpose	Access
Post Details	Post, likes and comments	Public/Logged-in for interactions
Admin Dashboard	System controls	Admin
Manage Users	User management	Admin
Manage Posts	Post moderation	Admin

7. MongoDB Database Design

Use one MongoDB database containing three primary collections: **users**, **posts** and **comments**.

Collection	Purpose	Main relationship
users	Stores account and role information	One user can create many posts and comments.
posts	Stores post content and engagement information	Each post belongs to one user and can have many commer
comments	Stores comments on posts	Each comment belongs to one post and one user.

7.1 User document structure

Field	Type	Required	Description
_id	ObjectId	Automatic	Unique MongoDB identifier.
name	String	Yes	User display/full name.
email	String	Yes	Unique login email; normalized form.
password	String	Yes	Hashed password; never store plain text.
profileImage	String	No	Profile image path/URL if implemented.
role	String	Yes	user or admin.
createdAt	Date	Automatic	Account creation timestamp.
updatedAt	Date	Automatic	Last update timestamp.

7.2 Post document structure

Field	Type	Required	Description
_id	ObjectId	Automatic	Unique post identifier.
title	String	Yes	Post title with reasonable maximum length.
content	String	Yes	Main post body.
image	String	No	Optional image path/URL.
author	ObjectId → User	Yes	Reference to creator.
likes	Array of ObjectId → User	No	Users who liked the post; prevents duplicate likes by user ID.
createdAt	Date	Automatic	Post creation timestamp.
updatedAt	Date	Automatic	Last modification timestamp.

7.3 Comment document structure

Field	Type	Required	Description
_id	ObjectId	Automatic	Unique comment identifier.
content	String	Yes	Comment text.
author	ObjectId → User	Yes	User who wrote the comment.
post	ObjectId → Post	Yes	Post on which the comment was written.

Field	Type	Required	Description
createdAt	Date	Automatic	Comment creation timestamp.
updatedAt	Date	Automatic	Last modification timestamp.

7.4 Relationships

Relationship	Meaning
User 1 → many Posts	One user can create many posts; each post has one author.
User 1 → many Comments	One user can write many comments; each comment has one author.
Post 1 → many Comments	One post can contain many comments.
User many ↔ many Posts through likes	A user can like many posts, and a post can be liked by many users.

Database rule: Agree on these structures before significant coding begins. Model changes after integration should be discussed by all four members.

8. API Structure

The group should agree on endpoint names before coding.

Module	Method	Endpoint	Purpose	Access
Auth	POST	/api/auth/register	Register user	Public
Auth	POST	/api/auth/login	Login	Public
Auth	GET	/api/auth/me	Current user	Logged-in
Users	GET	/api/users/profile	Own profile	Logged-in
Users	PUT	/api/users/profile	Update profile	Logged-in
Posts	GET	/api/posts	Browse posts	Public
Posts	GET	/api/posts/:id	Single post	Public
Posts	GET	/api/posts/my-posts	Own posts	Logged-in
Posts	POST	/api/posts	Create post	Logged-in
Posts	PUT	/api/posts/:id	Update post	Owner/Admin
Posts	DELETE	/api/posts/:id	Delete post	Owner/Admin
Likes	POST	/api/posts/:id/like	Like	Logged-in
Likes	DELETE	/api/posts/:id/like	Unlike	Logged-in
Comments	GET	/api/posts/:postId/comments	Get comments	Public
Comments	POST	/api/posts/:postId/comments	Add comment	Logged-in
Comments	PUT	/api/comments/:id	Edit comment	Owner/Admin
Comments	DELETE	/api/comments/:id	Delete comment	Owner/Admin
Admin	GET	/api/admin/users	List users	Admin
Admin	DELETE	/api/admin/users/:id	Delete user	Admin
Admin	GET	/api/admin/posts	List posts	Admin
Admin	DELETE	/api/admin/posts/:id	Moderate/delete post	Admin

Use a predictable response convention containing a success status, message and data. Error responses should provide a clear message and suitable HTTP status.

9. Four-Member Responsibility Plan

Member 1 — Authentication & User Management

Primary ownership: complete authentication and user module.

- Maintain User database structure.
- Registration and duplicate-email validation.
- Password hashing and verification.
- Login and JWT token generation.
- Authentication middleware and protected access.
- Current-user/profile retrieval and profile update.

- Login, Register and Profile pages.
- Authentication context/state and protected routes.
- Test invalid credentials, duplicate accounts and unauthorized requests.
- Document authentication flow for demonstration.

Main deliverables: User model, authentication backend, middleware, Login, Register, Profile, authentication state and protected routing.

Member 2 — Post Management / Core CRUD

Primary ownership: central Post Management module.

- Maintain Post database structure.
- Create, read, update and delete operations.
- Post ownership checks.
- Current-user post retrieval.
- Post search and basic filtering.
- Home, Post Details, Create Post, Edit Post and My Posts pages.
- PostCard and PostForm components.
- Loading, empty and error states.
- Complete CRUD and ownership testing.
- Document the post lifecycle from React to MongoDB.

Main deliverables: Post model, post controller/routes, CRUD pages, post components, search and testing.

Member 3 — Likes, Comments & Content Interaction

Primary ownership: social interaction functionality.

- Maintain Comment database structure.
- Like/unlike functionality.
- Prevent duplicate likes.
- Create/read/update/delete comments.
- Comment ownership checks.
- LikeButton, Comment, CommentForm and comment-list components.
- Integrate interactions into Post Details.
- Test likes, unlikes and comment operations.
- Document User–Post–Comment relationships.

Main deliverables: Comment model, interaction backend, interaction components and testing.

Member 4 — Dashboard, Admin & Integration Lead

Primary ownership: dashboard/admin features plus final project integration.

- Create base frontend structure, routing conventions and shared layout.
- Maintain Navbar and shared UI conventions.

- User dashboard and project statistics.
- Admin middleware and admin controls.
- Admin user management.
- Admin post moderation.
- Admin Dashboard, Manage Users and Manage Posts pages.
- Manage develop branch and review pull requests.
- Coordinate module integration and resolve conflicts.
- Run end-to-end testing after major merges.
- Maintain README and final documentation.
- Coordinate final demonstration.

Main deliverables: base frontend, Dashboard, Admin module, integration, testing, documentation and Git coordination.

10. Git and GitHub Workflow

Use one repository. Do not create four independent repositories.

Branch	Purpose
main	Stable final/demo version only.
develop	Integration branch for completed features.
feature/auth	Member 1 authentication/user work.
feature/posts	Member 2 post-management work.
feature/interactions	Member 3 likes/comments work.
feature/dashboard	Member 4 dashboard/admin work.

Rules for all four members

1. Pull latest develop before a new work session.
2. Work primarily inside your assigned module/files.
3. Commit small, meaningful changes.
4. Push feature branches regularly.
5. Create pull requests from feature branch to develop.
6. Do not directly push unfinished work to main.
7. Synchronize with develop after merges.
8. Communicate before changing shared files.
9. Never commit environment secrets or private database credentials.
10. Freeze major architectural changes before final demonstration and focus on testing.

Recommended merge sequence

Stage	Action
1	Create repository, base structure and develop branch.
2	Member 4 establishes shared frontend conventions and routing.
3	Member 1 completes authentication foundation.
4	Member 2 completes post CRUD against agreed schema/API.
5	Member 3 adds likes and comments.
6	Member 4 adds dashboard/admin.
7	Integrate modules into develop.
8	Run end-to-end testing.
9	Merge stable develop into main.
10	Prepare demonstration and final documentation.

11. Conflict-Reduction Rules

- Do not divide by MERN technology; divide by application feature.
- Freeze database structures early.
- Freeze API contracts early.
- Minimize shared-file editing.
- Keep one integration owner.
- Use pull requests for review.
- After every major merge, ensure the project starts and core functionality works.

12. Final Demonstration Plan

1. Project introduction: problem, objective and features.
2. MERN architecture: React → API → Express/Node → Mongoose → MongoDB.
3. Authentication: registration, login, password hashing and JWT.
4. Post CRUD: create, read, update and delete.
5. Interaction: like/unlike and comments.
6. Search/browse demonstration.
7. Dashboard demonstration.
8. Admin demonstration.
9. Database collections and relationships.
10. Each member explains their module and how it connects to the rest.
11. Brief Git/team integration explanation.

13. Final Testing Checklist

Area	Tests
Registration	Valid registration, duplicate email, invalid input.
Login	Correct password, wrong password, unknown account.
Authentication	Protected routes reject unauthenticated users.
Posts	Create, view, edit, delete and ownership checks.
Likes	Like, unlike and duplicate-like prevention.
Comments	Add, view, edit, delete and ownership checks.
Search	Relevant results and correct empty-state behavior.
Dashboard	Counts/statistics match actual data.
Admin	Admin-only features reject normal users.
UI	Navigation, loading, errors and responsive behavior.
Integration	Frontend, backend and database work together from a clean start.

14. Final Recommended Division — Summary

Member	Primary role	Main areas	Integration responsibility
1	Authentication & Users	MongoDB/Mongoose + Node/Express + React auth UI	Auth contract
2	Post Management	MongoDB/Mongoose + Node/Express + React post UI	Post contract
3	Likes & Comments	MongoDB/Mongoose + Node/Express + React interaction UI	Interaction contract
4	Dashboard, Admin & Integration	React + Node/Express + project-wide integration	Git, testing, final merge

Final recommendation: Feature-based division is preferable to assigning MongoDB, Express, React and Node.js individually. MERN is a connected stack, while features such as authentication and post management naturally cross the full stack. Feature ownership reduces conflicts, gives each member a meaningful contribution and makes the final demonstration stronger.

15. Optional Enhancements

- Image upload for posts.
- Post categories and tags.
- Pagination.
- Rich-text post editor.
- Bookmark/save post.
- Post reporting and moderation.
- Password reset.
- Dark/light theme.
- Deployment of frontend and backend.

Add these only after the core system is stable. A smaller, fully working project is better for a lab demonstration than a large unfinished project.